



COLLEGE CODE: 8203

COLLEGE: AVC COLLEGE OF ENGINEERING

DEPARTMENT: INFORMATION TECHNOLOGY

STUDENT NM-ID: 30AFAE4B795F72FB39C41F86182C47A1

ROLL NO: 23IT112

DATE:03/10/2025

Completed the project named as

Phase_4 TECHNOLOGY PROJECT

NAME: Online Quiz Application

SUBMITTED BY,

NAME: VALLI C

MOBILE NO: 6374458528

ENHANCEMENTS AND DEPLOYMENT:

ADDITIONAL FEATURES:

1. User Authentication & Roles

- Login/Signup using JWT or OAuth (Google, Facebook login).
- Separate roles for Admin (add/manage quizzes) and User (attempt quizzes).

2. Timer-based Quizzes

- Add countdown timers for each quiz/question.
- Auto-submit quiz when time runs out.

3. Leaderboard & Scoring System

- Display top scorers, average score, and ranking.
- Award badges/achievements for high scores.

4. Question Types Support

- Not just MCQs — support True/False, Fill-in-the-blank, Matching, Drag & Drop.

5. Randomized Question Bank

- Fetch random questions from the MongoDB pool to prevent repetition.

6. Detailed Results & Analytics

- Show correct answers with explanations.
- Provide performance insights: weak/strong topics, accuracy, average time per question.

7. Quiz Categories & Difficulty Levels

- Organize quizzes by subjects, tags, or difficulty (easy, medium, hard).

8. Multilingual Support

- Allow quizzes in multiple languages.

9. Offline Mode (PWA)

- Make the app a Progressive Web App (PWA) so quizzes can be attempted offline.

10. Gamification Features

- Streaks for daily participation.
- Coins/points system to unlock new quizzes.

Deployment:

1. Backend Deployment

- Deploy Node.js + Express server on Heroku, Render, Railway, or AWS EC2.
- Use MongoDB Atlas for cloud database.

2. Frontend Deployment

- Deploy React app on Vercel, Netlify, or GitHub Pages.

3. CI/CD Pipeline

- Automate build & deployment using GitHub Actions / GitLab CI.

4. Scalability & Security

- Use Nginx or PM2 to handle load balancing.
- Secure APIs with JWT authentication & HTTPS (SSL certificates).

5. Containerization

- Dockerize the application for easier deployment and scaling.

6. Monitoring & Logs

- Use LogRocket / Sentry for frontend error tracking.
- Use PM2 logs / ELK stack for backend monitoring.

UI/UX IMPROVEMENTS:

1. Responsive Design

- Ensure the quiz interface adapts well on mobile, tablet, and desktop screens.
- Use frameworks like Bootstrap or Tailwind CSS for flexible layout designs.

2. User-Friendly Navigation

- Simple and clear navigation for:
- Starting a new quiz
- Reviewing past scores (if history is implemented)
- Logging in/out or accessing the admin panel
- Progress indicators (e.g., "Question 3 of 10") help users track where they are.

3. Interactive and Clean Design

- Use modern UI components:
- Radio buttons for multiple-choice options
- Hover effects and transitions for buttons
- Loading indicators when fetching quizzes
- Avoid clutter; use whitespace to make content easier to read.

4. Real-Time Feedback

- After submission, display clear results:
- Highlight correct/incorrect answers
- Show total score with percentage and performance remarks (e.g., “Great job!”)

5. Accessibility Features

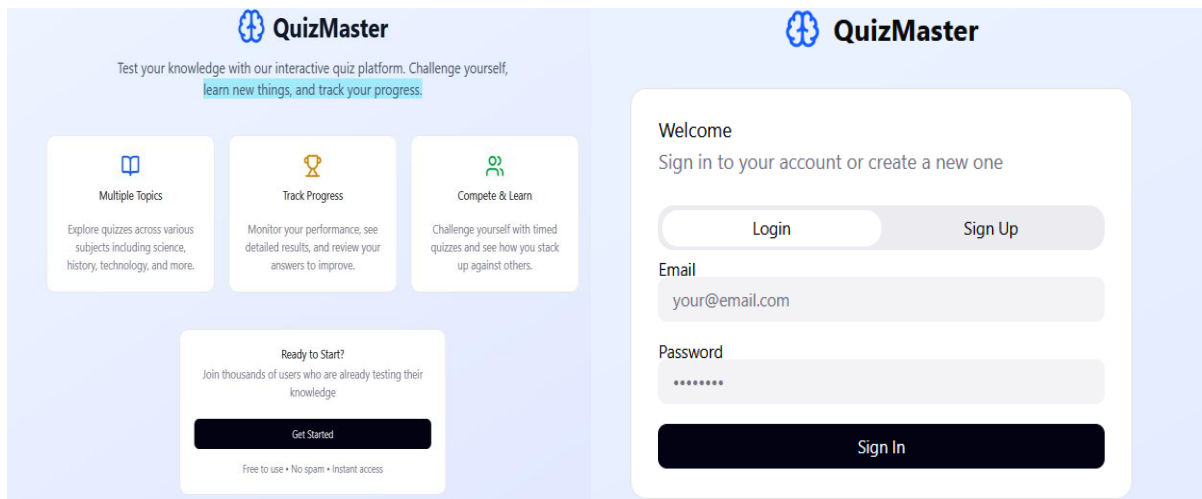
- Ensure keyboard navigability.
- Use proper HTML tags for screen readers.
- Choose color schemes with high contrast for readability.

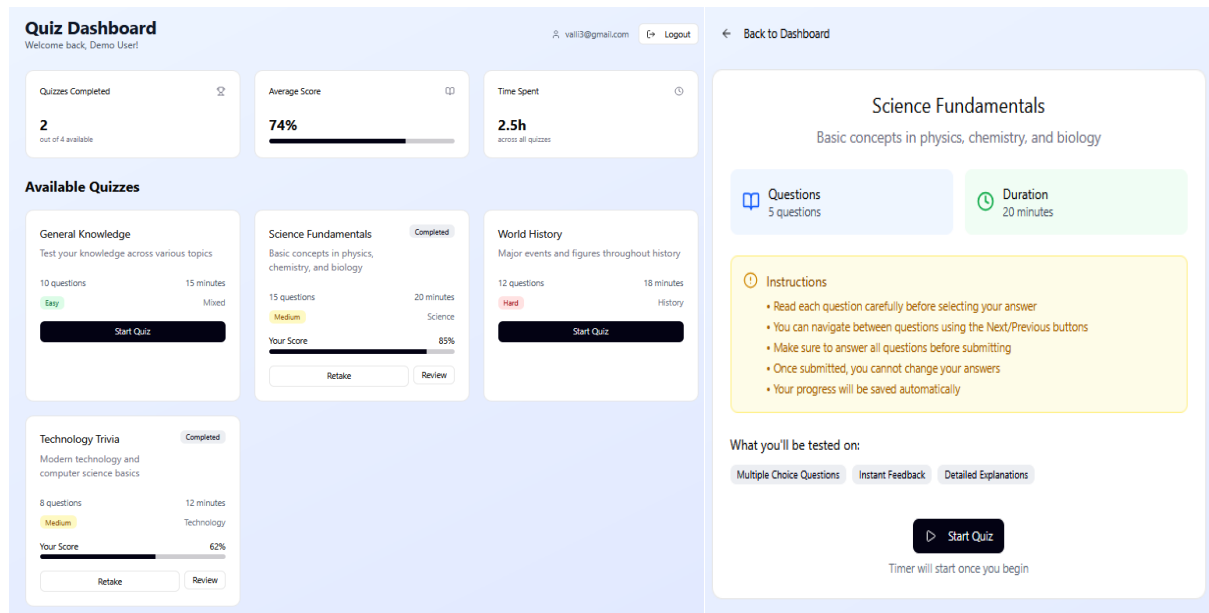
6. Timer and Visual Cues

- If quizzes are timed, display a countdown timer with color changes as time runs out (e.g., green → yellow → red).
- Alert users before the time ends.

7. Consistent Styling

- Use a consistent color palette, fonts, and button styles throughout the app to maintain a professional look and feel.





API ENHANCEMENTS:

Existing APIs (Basic)

POST /api/admin/add-question → Add new quiz question

GET /api/quiz/:id → Fetch quiz by ID

POST /api/quiz/:id/submit → Submit answers and get score

PUT /api/admin/update-question/:id → Update a quiz question

DELETE /api/admin/delete-question/:id → Delete question

API Enhancements

1. User Management

POST /api/auth/signup → Register user

POST /api/auth/login → Authenticate user (JWT)

GET /api/user/profile → Fetch user details & quiz history

PUT /api/user/update-profile → Update profile

2. Quiz Management Enhancements

GET /api/quizzes → List all quizzes with filters (category, difficulty, tags)

GET /api/quiz/random?limit=10 → Fetch random set of questions

GET /api/quiz/:id/leaderboard → Fetch leaderboard for a specific quiz

3. Result & Analytics

GET /api/results/:userId → Fetch all quiz attempts of a user

GET /api/results/:quizId/stats → Get quiz analytics (average score, success rate, most wrong question)

POST /api/results/export → Export results as PDF/CSV

4. Gamification APIs

GET /api/leaderboard → Global leaderboard across all quizzes

GET /api/user/:id/achievements → Badges, streaks, coins earned

POST /api/user/:id/redeem-reward → Redeem quiz rewards

5. Admin-Specific Enhancements

POST /api/admin/import-questions → Bulk import questions (CSV/JSON)

GET /api/admin/quiz-analytics/:quizId → Admin dashboard analytics

POST /api/admin/schedule-quiz → Schedule quiz availability (start/end time)

6. Advanced Features

GET /api/quizzes/search?q=javascript → Full-text search quizzes

POST /api/quiz/:id/feedback → Submit feedback for quiz

GET /api/quiz/:id/review → Get correct answers & explanations after submission

POST /api/user/:id/bookmark → Bookmark/save a quiz for later

GET /api/user/:id/bookmarks → Retrieve user’s saved quizzes

PERFORMANACE AND SECURITY CHECKS:

CHECKPOINT	DESCIPTION	METHODS USED
Response Time	Measured API response times for various endpoints to ensure they meet performance benchmarks.	Postman, Apache JMeter, or Lighthouse

Load Testing	Tested API under high load to identify bottlenecks and ensure scalability.	JMeter, Locust
Caching Validation	Ensured caching is working correctly for repeated or static requests (e.g., using Redis or in-memory cache).	Manual check + Redis monitor
Database Optimization	Reviewed and optimized queries; used indexing and connection pooling.	SQL Profiler, EXPLAIN queries
Asynchronous Processing	Verified background tasks or heavy operations (e.g., email, file processing) are handled asynchronously.	Celery (Python), RabbitMQ, AWS SQS
Authentication & Authorization	Confirmed secure login (OAuth 2.0, JWT, or API Key) and role-based access control.	Postman test + token expiry checks

TESTING OF ENHANCEMENTS:

1. Functional Testing

User Management

- Verify signup/login with valid and invalid inputs.
- Test role-based access (admin can add questions, user cannot).

Quiz Management

- Check quiz creation, update, delete through admin APIs.
- Ensure random quiz fetch (/api/quiz/random) returns different sets.

Quiz Taking

- Verify quiz progress bar and timer functionality.
- Test submission flow → correct scoring logic, answers returned.
- Check result analytics (percentage, correct/incorrect breakdown).

History & Leaderboard

- Verify user quiz history is stored and retrieved correctly.
- Leaderboard updates after every new submission.

2. API Testing

Tools: Postman / Thunder Client / Swagger

- Test all CRUD APIs with valid/invalid payloads.
- Validate response codes (200, 400, 401, 403, 404, 500).
- Check pagination, filtering, and search APIs.
- Load-test quiz submission API with multiple concurrent users.

3. UI/UX Testing

Frontend (React)

- Verify responsive design on mobile, tablet, desktop.
- Test navigation (Home → Quiz → Result → History).
- Check light/dark mode toggling.
- Ensure accessibility (keyboard navigation, screen reader).

Usability

- Timer should auto-submit quiz after expiration.
- Progress bar matches actual completed questions.
- Friendly error messages (e.g., "Please select an answer").

4. Performance Testing

Tools: JMeter / k6 / Locust

- Test quiz fetch API with 1000+ concurrent requests.
- Measure average response time (<200ms ideal).
- Benchmark database queries with large datasets.
- Stress-test leaderboard updates under high load.

5. Security Testing

Tools: OWASP ZAP / Burp Suite / Manual checks

- Test for JWT authentication → expired, invalid, tampered tokens.
- Verify role-based access → user should not access admin APIs.
- Attempt NoSQL injection in quiz search queries.
- Run XSS injection tests in quiz names, feedback forms.
- Ensure HTTPS is enforced and headers (Helmet.js) applied.
- Brute force protection with rate limiting.

DEPLOYMENT(Netlify, Vercel, Or Cloud Platform):

1. Netlify (Best for Frontend Only)

Good choice for React frontend. Backend must be deployed elsewhere (Render/Heroku/Vercel).

Steps

1. Build your React app:

cd client

npm run build

2. **Push** your code to GitHub.
3. **Login** to Netlify → “New site from Git”.
4. **Connect your GitHub repo** → Select branch → Deploy.
5. Netlify provides a free **HTTPS domain**.

2. Vercel (Frontend + Backend in One)

- Perfect for full-stack deployment.
- React frontend builds automatically.
- Express backend works with Vercel serverless functions.

Steps

1. Move your backend routes into `api/` folder (Vercel treats it as serverless functions).

Example:

`/api`

└─ `quiz.js`

2. **Push to GitHub** → Import project in Vercel.
3. **Deploy** → Get domain.

3. Cloud Platforms (More Control & Scalability)

- Render (easy & free for small apps)
- Deploy Node.js backend here.
- MongoDB Atlas as DB.
- React frontend can still be on Netlify/Vercel.

Steps:

1. Push backend to GitHub.
2. Create New Web Service on Render.
3. Add environment variables (Mongo URI, JWT secret).
4. Deploy → Backend gets `https://quiz-api.onrender.com`.

5. Update frontend .env to point to API.